Two services, one signed-in user

This report follows Félix's two assigned services, User and DailyMealPlan, from the original JSON application into separate .NET processes. The calorie-goal feature shipped through PR #13. The meal-planning work is tracked in issue #14. The examples below describe the implementation and distinguish automated evidence from human review.

Where each responsibility lives

The User domain enforces a positive whole-number calorie goal. Its data project maps one profile to a validated identity and stores the goal in SQLite. The API exposes the current user's goal, so a caller never needs to send an account ID. The React client signs in, reads the goal over HTTP and sends updates to the API.

DailyMealPlan has a separate database and migration. Its domain adds the calories for breakfast, lunch and dinner, then multiplies each snack's calorie value by its quantity. The API resolves recipe and ingredient IDs through HTTP before saving the plan and snacks together. An unknown ingredient identifies the offending field. If a lookup service goes offline during an edit, the previous plan stays available.

> A regression fixture uses three 40-calorie meals, half a 40-calorie recipe and two 40-calorie ingredient portions. The same inputs must produce 220 calories after a save and an API restart.

How the clients and services share identity

Félix selected ASP.NET Core Razor Pages for the meal planner after the team discussed its frontend preference. Razor Pages obtains an access token with authorization code and PKCE and calls the meal-plan API through HttpClient. Each API checks signature, issuer, audience and expiry. A read scope permits viewing, and a write scope permits changes. Both the issuer and subject identify the owner. Guessing another user's plan ID returns 404.

The tracked legacy data contains no meal plans. A deployment with existing JSON records needs a backup and a verified mapping from old account IDs to issuer and subject before migration. The demo's HTTP lookup contract also needs confirmation from the Recipe and Ingredient owners before deployment.

What the development gates caught

Division of work

The repository's earlier PRs assign Recipe and Ingredient to Sarah, Grocery List and Storage to Zachary, and User and DailyMealPlan to Félix. For these two stories, Félix owned the backend and connected screens. Zachary approved and merged the User pull request. This report covers the evidence available for those stories. Each service owner supplies the final status of their own work for the team submission.

Where agents helped, and where checks mattered

The repository Planner produced a layer-by-layer plan in Visual Studio. Before implementation, Félix requested separate projects and a calorie-goal-only response for User, then approved the meal-plan plan and selected its frontend. The Domain, Data, API and Frontend agents produced focused changes. Their project boundaries made it possible to compile a domain without ASP.NET Core or EF Core and to exercise the API independently of the screen.

Integration exposed details that generated code alone had missed. The initial meal-plan migration snapshot referenced an unavailable SQLite API. Regenerating it with the installed EF tooling produced a migration the integration tests could apply. The initial Razor client requested a profile scope the local realm did not advertise. Exercising the real sign-in flow identified that mismatch. The final client requests the scopes needed for meal planning.

Evidence for human signoff

On PR #13, Félix recorded saving and refreshing a valid goal, rejection of an invalid value, and the sign-in check. He separately described reading the permission checks and the two-user restart test. Zachary's approval records the independent review. Automated review also found that the screen could show an empty goal after a failed read. The follow-up distinguishes an unknown result from a successful response containing no goal.

The meal-plan suite creates two users, attempts access to another owner's plan, checks failures from both token validation and missing scopes, and reopens SQLite through a new API host. A browser walkthrough exercises a different boundary: the local issuer, Razor forms and HTTP API working together. The author and peer record their own decisions on the meal-plan PR. The team submission still requires the board evidence and the Azure DevOps milestone PR link specified in the handout.